

Classification Level: Top secret () Secret ()
Internal () Public (√)

ROCKIVA SDK Developer Guide

(Technology Department, Graphic Display
Platform Center)

Mark: [] Editing [√] Released	Version:	V2.0.0BETA
	Author	AI Team
	Completed Date	2024-10-14
	Reviewer	Xiong Wei
	Reviewed Date	2024-10-14

福州瑞芯微电子股份有限公司
Fuzhou Rockchips Semiconductor Co . , Ltd
(版本所有,翻版必究)

Revision History

Version	Modifier	Date	Modify Description	Reviewer
V2.0.0BET A	Yang Huacong	2024-10- 14	Initial version	Xiong Wei

Table of Contents

1	Overview	4
2	SDK Model Description	5
2.1	Detection Model Description	5
3	SDK Development Instructions	6
3.1	General Module	6
3.1.1	Function Description	6
3.1.2	API Reference	9
3.2	Detection module	31
3.2.1	Function Description	31
3.2.2	API Reference	34
3.3	Perimeter module	36
3.3.1	Function Description	36
3.3.2	API Reference	41

1 Overview

ROCKIVA SDK is a set of Intelligent Video Analysis (IVA) Algorithm SDK, capable of being embedded in products that utilize video streams from NVRs, IPCs, and other cameras, providing support for AI intelligent algorithms. The main functions of the SDK include object detection, perimeter functions, face capture, etc.:

- > Object detection function: detects persons, faces, motor vehicles, non-motor vehicles, and pet in the image, returning the location of the targets;
- > Perimeter function: supports perimeter functions such as area intrusion, boundary crossing detection, area entry, and area exit;

2 SDK Model Description

2.1 Detection Model Description

To meet the needs of various product types, the SDK provides multiple object detection models, which users can select according to their needs.

You can directly configure RockIvaInitParam 's detModelName to the object detection model file name, and the framework will automatically load the corresponding model file from the configured model directory path.

1) All-in-One Detection Model

Detection Categories: Person, Face, Motor Vehicle, Non-motor Vehicle, License Plate, Pet, Human Head

Detection Model File	Suitable Resolution	Memory Usage MB	Flash Occupancy MB (Uncompressed)	Time Taken ms
iva_ object_detection_v3_cls8_896x512.data	960x540	9.26	4.8	107
iva_ object_detection_v3_cls8_640x384.data	640x360	7.11	4.8	67

2) Face, Human, Pet

Detection Categories: Person, Face, Pet

Detection Model	Suitable Resolution	Memory Usage MB	Flash Occupancy MB (Uncompressed)	Time Taken ms
iva_ object_detection_v3_pfp_nn_896x512.data	960x540	7.71	3.2	79
iva_ object_detection_v3_pfp_nn_640x384.data	640x360	5.57	3.2	52
iva_ object_detection_v3_pfp_lite_896x512.data	960x540	5.73	1.1	91
iva_ object_detection_v3_pfp_lite_640x384.data	640x360	4.16	1.1	58

3) Human, Vehicle, Pet

Detection Categories: Person, Face, Motor Vehicle, Pet

Detection Model	Suitable Resolution	Memory Usage MB	Flash Occupancy MB (Uncompressed)	Time Taken ms
iva_ object_detection_v3_pfcv_nn_896x512.data	960x540	7.71	3.2	79
iva_ object_detection_v3_pfcv_nn_640x384.data	640x360	5.57	3.2	52
iva_ object_detection_v3_pfcv_lite_896x512.data	960x540	5.73	1.1	91
iva_ object_detection_v3_pfcv_lite_640x384.data	640x360	4.16	1.1	58

5) Human, Vehicle

Detection Categories: Person, Motor Vehicle

Detection Model	Suitable Resolution	Memory Usage MB	Flash Occupancy MB (Uncompressed)	Time Taken ms
iva_object_detection_v3_pv_lite_896x512.data	960x540	5.73	1.1	91
iva_object_detection_v3_pv_lite_640x384.data	640x360	4.16	1.1	58

3 SDK Development Instructions

The ROCKIVA SDK first obtains target information such as person, face, motor vehicles, and non-motor vehicles from images based on object detection and tracking algorithms, and then further conducts intelligent analysis based on the results of object detection and tracking. The currently supported intelligent analysis functions include perimeter, face, etc. Each intelligent analysis function can be independently enabled or disabled, and they can also be used simultaneously.

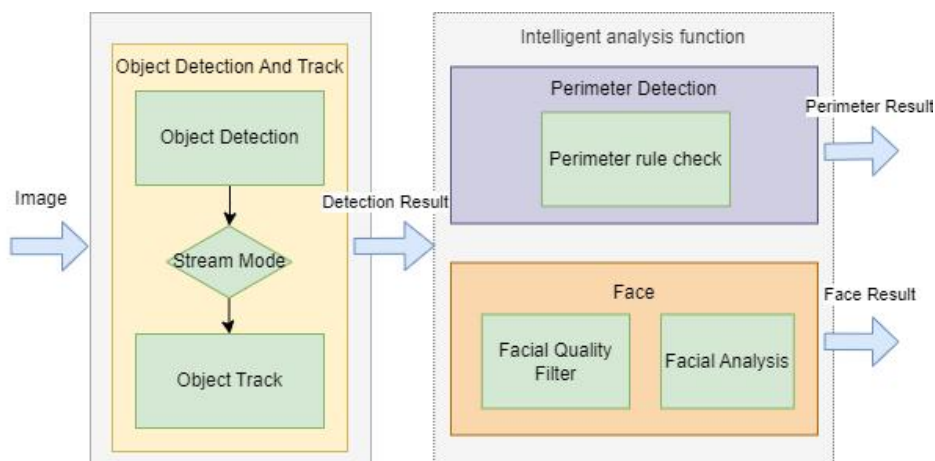


Figure 3-1 ROCKIVA SDK Overall Process

3.1 General Module

3.1.1 Function Description

The ROCKIVA SDK interface operates in asynchronous mode, with the basic operations being configuration initialization, followed by frame pushing at a certain frame rate, obtaining results in the callback function, releasing image frame memory, and finally releasing the instance if it is no longer needed.

The basic calling process of the SDK is shown in Figure 3-2.

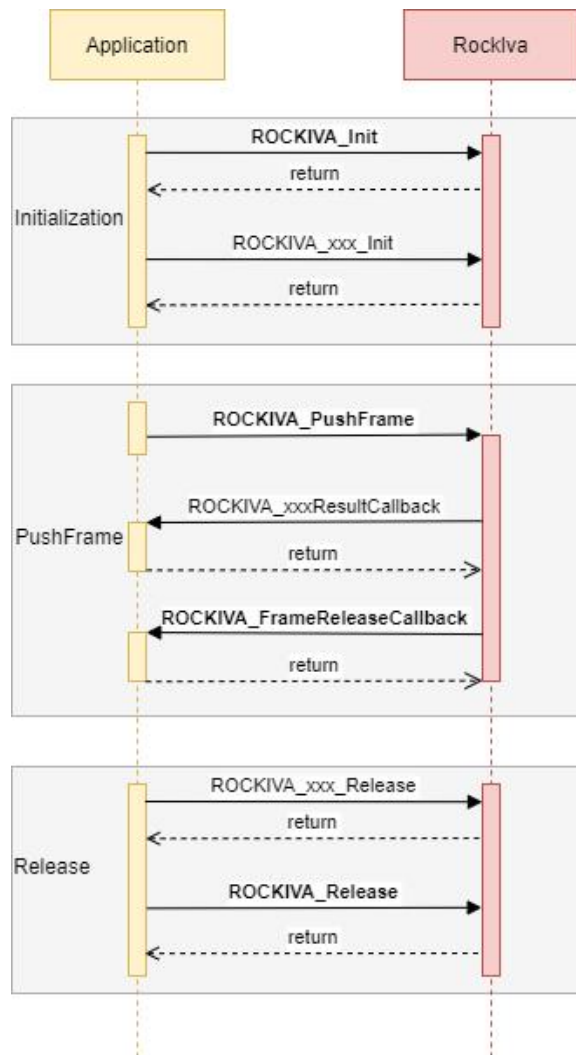


Figure 3-2 SDK Basic Calling Process

After configuration initialization, the SDK will create threads internally to asynchronously process each frame. Each image frame pushed by the application will be placed into a cache queue for the processing thread.

The callback function `ROCKIVA_FrameReleaseCallback` is used to notify the application that an image frame can be released. The SDK will determine when the image frame is no longer needed and notify the external application that it can be released.

Reference code:

```

RockIvaRetCode ret;
RockIvaHandle handle;

// Release callback function
void FrameReleaseCallback(const RockIvaReleaseFrames*
releaseFrames, void* userdata)
{
    // Handle release
}

// Module callback function, such as detection module
void DetResultCallback(const RockIvaDetectResult* result, const
RockIvaExecuteStatus status, void* userdata)
{
    // Handle results
}

// Initialize I VA instance
RockIvaInitParam commonParams ;
memset (&commonParams, 0, sizeof(RockIvaInitParam));
commonParams.detModel Name = iva_
object_detection_v3_cls8_896x512.data ;    // Set Detection Model
ret = ROCKIVA_Init(&handle, ROCKIVA_MODE_VIDEO, &commonParams,
NULL);
// Set Callback Function
ROCKIVA_SetFrameReleaseCallback(handle, framerelease_callback);

// Initialize required IVA modules, such as the detection
module
ret = ROCKIVA_DETECT_Init(handle, &detParams, DetResultCallback);

// Process Image Frame
while(running) {
    RockIvaImage image;
    memset(&image, 0, sizeof(RockIvaImage));
    // Capture Frame
    ...
    // Set Frame Data
    image->info.width = image_width;
    image->info.height = image_height;
}

```



```

    image->info.format = image_pixel_format;
    image->dataAddr = image_buffer_virtual_address;
    image->dataFd = image_buffer_fd;
    image->frameId = frame_index;

    ...

    / / Send Frame
    ROCKIVA_PushFrame(handle, image, NULL);
}

// Wait for all frames to be processed
usleep(5*1000*1000);
/ / Destroy Module
ROCKIVA_DETECT_Release(handle)
/ / Destroy IVA Instance
ROCKIVA_Release(handle);

```

3.1.2 API Reference

3.1.2.1 Interface

Interface	Description
ROCKIVA_Init	Algorithm SDK Initialization Configuration
ROCKIVA_Release	Algorithm SDK Destruction and Release
ROCKIVA_WaitFinish	Wait for Algorithm Frame Processing to Complete
ROCKIVA_SetFrameReleaseCallback	Set Frame Release Callback Function
ROCKIVA_FrameReleaseCallback	Frame Release Callback Function
ROCKIVA_PushFrame	Input Image Frame
ROCKIVA_PushFrameWithRoi	Specific Valid Area of Input Image Frame
ROCKIVA_GetVersion	Get SDK Version

3.1.2.1.1 ROCKIVA_Init

[Function]

Initialize Algorithm SDK, initialize the passed handle

[Declaration]

```
RockIvaRetCode ROCKIVA_Init (RockIvaHandle* handle,
RockIvaWorkMode mode,
                                const RockIvaInitParam* param, void
*userdata);
```

[Parameters]

Parameter Name	Input /Output	Description
handle	Input	Instance handle to be initialized
mode	Input	Working mode, refer to RockIvaWorkMode definition
param	Input	Initialization parameters, refer to RockIvaInitParam definition
userdata	Input	User-defined data

[Return Value]

RockIvaRetCode

3.1.2.1.2 **ROCKIVA_Release**

[Function]

Algorithm SDK Destruction and Release

[Declaration]

```
RockIvaRetCode ROCKIVA_Release (RockIvaHandle handle);
```

[Parameters]

Parameter Name	Input /Output	Description
handle	Input	Instance handle to be released

[Return Value]

RockIvaRetCode

3.1.2.1.3 **ROCKIVA_SetFrameReleaseCallback**

[Function]

Set Frame Release Callback Function

[Declaration]

```
RockIvaRetCode ROCKIVA_SetFrameReleaseCallback (RockIvaHandle
handle, ROCKIVA_FrameReleaseCallback callback) ;
```

[Parameters]

Parameter Name	Input /Output	Description
handle	Input	Instance Handle
callback	Input	Frame Release Callback Function

[Return Value]

RockIvaRetCode

[Description]

It is recommended to store the frame corresponding information in the extData pointer of RockIvaImage, and then retrieve it for release when freeing

3.1.2.1.4 ROCKIVA_FrameReleaseCallback

[Function]

Frame Release Callback Function

[Declaration]

```
typedef void (* ROCKIVA_FrameReleaseCallback ) (const
RockIvaReleaseFrames* releaseFrames, void *userdata);
```

[Parameters]

Parameter Name	Input /Output	Description
releaseFrames	Input	Releasable frame list, see RockIvaReleaseFrames definition
userdata	Input	User-defined data

[Return Value]

RockIvaRetCode

3.1.2.1.5 ROCKIVA_PushFrame

[Function]

Input Image Frame

[Declaration]

```
RockIvaRetCode ROCKIVA_PushFrame (RockIvaHandle handle, const
RockIvaImage* inputImg, const RockIvaFrameExtraInfo* extraInfo);
```

[Parameters]

Parameter Name	Input /Output	Description
handle	Input	Instance Handle
inputImg	Input	Input image frame, see RockIvaImage definition
extraInfo	Input	Additional information (can be NULL if not needed), see RockIvaFrameExtraInfo definition

[Return Value]

RockIvaRetCode

3.1.2.1.6 ROCKIVA_SetParam

[Function]

Set Parameters

[Declaration]

```
RockIvaRetCode ROCKIVA_SetParam (RockIvaParams* params, const
char* key, const char* value);
```

[Parameters]

Parameter Name	Input /Output	Description
params	Input	Parameters, see RockIvaParams definition
key	Input	Keyword
value	Input	Value

[Return Value]

RockIvaRetCode

3.1.2.1.7 ROCKIVA_GetVersion

[Function]

Get SDK Version Number

[Declaration]

```
RockIvaRetCode ROCKIVA_GetVersion (const uint32_t maxLen, char*
version);
```

[Parameters]

Parameter Name	Input /Output	Description
maxLen	Input	Buffer size for storing version number
version	Input	Buffer address for version number

[Return Value]

RockIvaRetCode

3.1.2.1.8 ROCKIVA_Yuv2Jpeg

[Function]

Callback function for encoding image to JPEG image

[Declaration]

```
int (*ROCKIVA_Yuv2Jpeg)(RockIvaImage *inputImg, RockIvaImage
*outputImg, void *userdata);
```

[Parameters]

Parameter Name	Input /Output	Description
inputImg	Input	Image to be encoded
outputImg	Output	Encoded image (Note: outputImg.dataAddr needs to be allocated and freed by the user externally; when the internal memory for this image is no longer used, it will be reported in the snapshot callback function ROCKIVA_FrameReleaseCallback)
userdata	Input	Custom pointer for initialization settings

[Return Value]

RockIvaRetCode

3.1.2.2 Enumeration

Enumeration	Description
RockIvaLogLevel	Log print level
RockIvaRetCode	Function return error code
RockIvaExecuteStatus	Execution Callback Result Status Code
RockIvaImageFormat	Image Pixel Format
RockIvaImageTransform	Input Image Rotation Mode
RockIvaMemType	Memory Type
RockIvaObjectType	Target Type
RockIvaObjectState	Target Tracking Status
RockIvaDetModel	Pre-stage Object Detection Algorithm Model
RockIvaWorkMode	Working Mode
RockIvaCameraType	Camera Type

3.1.2.2.1 RockIvaLogLevel

[Function]

Log print level

[Declaration]

```
typedef enum {
    ROCKIVA_LOG_ERROR = 0,
    ROCKIVA_LOG_WARN = 1,
    ROCKIVA_LOG_DEBUG = 2,
    ROCKIVA_LOG_INFO = 3,
    ROCKIVA_LOG_TRACE = 4
} RockIvaLogLevel;
```

[Member]

Member	Description
ROCKIVA_LOG_ERROR	Error Level (Default)
ROCKIVA_LOG_WARN	Warning Level
ROCKIVA_LOG_DEBUG	Debug Level
ROCKIVA_LOG_INFO	Information Level
ROCKIVA_LOG_TRACE	Tracking Call Level

[Description]

The larger the log level value, the more logs will be printed.

3.1.2.2.2 RockIvaRetCode

[Function]

Function Call Return Error Code

[Declaration]

```
typedef enum {
    ROCKIVA_RET_SUCCESS = 0,
    ROCKIVA_RET_FAIL = -1,
    ROCKIVA_RET_NULL_PTR = -2,
    ROCKIVA_RET_INVALID_HANDLE = -3,
    ROCKIVA_RET_LICENSE_ERROR = -4,
    ROCKIVA_RET_UNSUPPORTED = -5,
    ROCKIVA_RET_STREAM_SWITCH = -6,
    ROCKIVA_RET_BUFFER_FULL = -7,
} RockIvaRetCode;
```

[Member]

Member	Description
ROCKIVA_RET_SUCCESS	Success
ROCKIVA_RET_FAIL	Failure
ROCKIVA_RET_NULL_PTR	Null pointer
ROCKIVA_RET_INVALID_HANDLE	Invalid handle
ROCKIVA_RET_LICENSE_ERROR	License error
ROCKIVA_RET_UNSUPPORTED	Not supported
ROCKIVA_RET_STREAM_SWITCH	Stream switching
ROCKIVA_RET_BUFFER_FULL	Buffer area full

3.1.2.2.3 RockIvaExecuteStatus

[Function]

Execution Callback Result Status Code

[Declaration]

```
typedef enum {
    ROCKIVA_SUCCESS = 0,
    ROCKIVA_UNKNOWN = 1,
    ROCKIVA_NULL_PTR = 2,
    ROCKIVA_ALLOC_FAILED = 3,
    ROCKIVA_INVALID_INPUT = 4,
    ROCKIVA_EXECUTE_FAILED = 6,
    ROCKIVA_NOT_CONFIGURED = 7,
    ROCKIVA_NO_CAPACITY = 8,
    ROCKIVA_BUFFER_FULL = 9,
    ROCKIVA_LICENSE_ERROR = 10,
    ROCKIVA_JPEG_DECODE_ERROR = 11,
    ROCKIVA_DECODER_EXIT = 12,
} RockIvaExecuteStatus;
```

[Member]

Member	Description
ROCKIVA_SUCCESS	Execution result normal
ROCKIVA_UNKNOWN	Unknown error type
ROCKIVA_NULL_PTR	Operation on null pointer
ROCKIVA_ALLOC_FAILED	Failed to allocate space
ROCKIVA_INVALID_INPUT	Invalid input
ROCKIVA_EXECUTE_FAILED	Internal execution error
ROCKIVA_NOT_CONFIGURED	Unconfigured type
ROCKIVA_NO_CAPACITY	Business has reached capacity
ROCKIVA_BUFFER_FULL	Buffer area full
ROCKIVA_LICENSE_ERROR	License exception
ROCKIVA_JPEG_DECODE_ERROR	Decoding error
ROCKIVA_DECODER_EXIT	Decoder internal exit

3.1.2.2.4 RockIvaImageFormat

[Function]

Image pixel format type

[Declaration]

```
typedef enum {  
    ROCKIVA_IMAGE_FORMAT_GRAY8 = 0,  
    ROCKIVA_IMAGE_FORMAT_RGB888,  
    ROCKIVA_IMAGE_FORMAT_BGR888,  
    ROCKIVA_IMAGE_FORMAT_RGBA8888,  
    ROCKIVA_IMAGE_FORMAT_BGRA8888,  
    ROCKIVA_IMAGE_FORMAT_YUV420P_YU12,  
    ROCKIVA_IMAGE_FORMAT_YUV420P_YV12,  
    ROCKIVA_IMAGE_FORMAT_YUV420SP_NV12,  
    ROCKIVA_IMAGE_FORMAT_YUV420SP_NV21,  
    ROCKIVA_IMAGE_FORMAT_JPEG,  
} RockIvaImageFormat;
```

[Member]

Member	Description
ROCKIVA_IMAGE_FORMAT_GRAY8	Single-channel grayscale image
ROCKIVA_IMAGE_FORMAT_RGB888	Three-channel color image
ROCKIVA_IMAGE_FORMAT_BGR888	Three-channel color image
ROCKIVA_IMAGE_FORMAT_RGBA8888	Four-channel color image
ROCKIVA_IMAGE_FORMAT_BGRA8888	Four-channel color image
ROCKIVA_IMAGE_FORMAT_YUV420P_YU12	YUV420P_YU12
ROCKIVA_IMAGE_FORMAT_YUV420P_YV12	YUV420P_YV12
ROCKIVA_IMAGE_FORMAT_YUV420SP_NV12	YUV420SP_NV12
ROCKIVA_IMAGE_FORMAT_YUV420SP_NV21	YUV420SP_NV21
ROCKIVA_IMAGE_FORMAT_JPEG	JPEG image (input image does not support this format)

[Notes]

None

3.1.2.2.5 RockIvaImageTransform

[Function]

Input Image Rotation Mode

[Declaration]

```
typedef enum {
    ROCKIVA_IMAGE_TRANSFORM_NONE = 0x00,
    ROCKIVA_IMAGE_TRANSFORM_FLIP_H = 0x01,
    ROCKIVA_IMAGE_TRANSFORM_FLIP_V = 0x02,
    ROCKIVA_IMAGE_TRANSFORM_ROTATE_90 = 0x04,
    ROCKIVA_IMAGE_TRANSFORM_ROTATE_180 = 0x03,
    ROCKIVA_IMAGE_TRANSFORM_ROTATE_270 = 0x07,
} RockIvaImageFormat;
```

[Member]

Member		Description
ROCKIVA_IMAGE_TRANSFORM_NONE		Normal
ROCKIVA_IMAGE_TRANSFORM_FLIP_H		Horizontal flip
ROCKIVA_IMAGE_TRANSFORM_FLIP_V		Vertical flip
ROCKIVA_IMAGE_TRANSFORM_ROTATE_90		Clockwise 90 degrees
ROCKIVA_IMAGE_TRANSFORM_ROTATE_180		Clockwise 180 degrees
ROCKIVA_IMAGE_TRANSFORM_ROTATE_270		Clockwise 270 degrees

3.1.2.2.6 RockIvaObjectType

[Function]

Detected target category

[Declaration]

```
typedef enum {
    ROCKIVA_OBJECT_TYPE_NONE = 0,
    ROCKIVA_OBJECT_TYPE_PERSON = 1,
    ROCKIVA_OBJECT_TYPE_VEHICLE = 2,
    ROCKIVA_OBJECT_TYPE_NON_VEHICLE = 3,
    ROCKIVA_OBJECT_TYPE_FACE = 4,
    ROCKIVA_OBJECT_TYPE_HEAD = 5,
    ROCKIVA_OBJECT_TYPE_PET = 6,
    ROCKIVA_OBJECT_TYPE_MOTORCYCLE = 7,
    ROCKIVA_OBJECT_TYPE_BICYCLE = 8,
    ROCKIVA_OBJECT_TYPE_PLATE = 9,
    ROCKIVA_OBJECT_TYPE_BABY = 10,
    ROCKIVA_OBJECT_TYPE_MAX
} RockIvaObjectType;
```

[Member]

Member	Description
ROCKIVA_OBJECT_TYPE_NONE	Unknown
ROCKIVA_OBJECT_TYPE_PERSON	Person
ROCKIVA_OBJECT_TYPE_VEHICLE	Motor Vehicle
ROCKIVA_OBJECT_TYPE_NON_VEHICLE	Non-motor vehicle
ROCKIVA_OBJECT_TYPE_FACE	Face
ROCKIVA_OBJECT_TYPE_HEAD	Human Head
ROCKIVA_OBJECT_TYPE_PET	Cat, Dog
ROCKIVA_OBJECT_TYPE_MOTORCYCLE	Electric bicycle
ROCKIVA_OBJECT_TYPE_BICYCLE	Bicycle
ROCKIVA_OBJECT_TYPE_PLATE	License Plate
ROCKIVA_OBJECT_TYPE_BABY	Infant

3.1.2.2.7 RockIvaObjectState

[Function]

Target Tracking Status

[Declaration]

```
typedef enum {
    ROCKIVA_OBJECT_STATE_NONE,
    ROCKIVA_OBJECT_STATE_FIRST,
    ROCKIVA_OBJECT_STATE_TRACKING,
    ROCKIVA_OBJECT_STATE_LOST,
    ROCKIVA_OBJECT_STATE_DISPEAR,
} RockIvaObjectState;
```

[Member]

Member	Description
ROCKIVA_OBJECT_STATE_NONE	Target Status Unknown
ROCKIVA_OBJECT_STATE_FIRST	Target First Appeared
ROCKIVA_OBJECT_STATE_TRACKING	During Object Detection Tracking
ROCKIVA_OBJECT_STATE_LOST	Object Detection Lost
ROCKIVA_OBJECT_STATE_DISPEAR	Target Disappeared

3.1.2.2.8 RockIvaDetModel

[Function]

Pre-stage Object Detection Algorithm Model

[Declaration]

```
typedef enum {
    ROCKIVA_DET_MODEL_NONE = 0,
    ROCKIVA_DET_MODEL_CLS7,
    ROCKIVA_DET_MODEL_PFCP,
    ROCKIVA_DET_MODEL_PFP,
    ROCKIVA_DET_MODEL_PHS,
    ROCKIVA_DET_MODEL_PHCP,
    ROCKIVA_DET_MODEL_PERSON,
    ROCKIVA_DET_MODEL_NONVEHICLE,
    ROCKIVA_DET_MODEL_FHS,
    ROCKIVA_DET_MODEL_PHCN,
    ROCKIVA_DET_MODEL_CLS8,
} RockIvaDetModel;
```

[Member]

Member	Description
ROCKIVA_DET_MODEL_NONE	Not Set, No Need for Preceding Object Detection
ROCKIVA_DET_MODEL_CLS7	Model File: object_detection_ipc_cls7.data Detection Categories: Person, Face, Motor Vehicle, License Plate, Non-motor Vehicle, Pet
ROCKIVA_DET_MODEL_PFCP	Model File: object_detection_pfcps.data Detection Categories: Person, Face, Motor Vehicle, Pet
ROCKIVA_DET_MODEL_PFP	Model File: object_detection_pfp.data Detection Categories: Person, Face, Pet
ROCKIVA_DET_MODEL_PHS	Model File: object_detection_phs.data Detection Categories: Person, Head and Shoulders
ROCKIVA_DET_MODEL_PHCP	Model File: object_detection_phcps.data Detection Categories: Person, Head and Shoulders, Motor Vehicle, Pet
ROCKIVA_DET_MODEL_PERSON	Model File: object_detection_person.data Detection Category: Person
ROCKIVA_DET_MODEL_NONVEHICLE	Model File: object_detection_motorcycle.data Detection Category: Electric Vehicle
ROCKIVA_DET_MODEL_FHS	Model File: object_detection_fhs.data Detection Category: Face, Head and Shoulders
ROCKIVA_DET_MODEL_PHCN	Model File: object_detection_phcns.data Detection Category: Person, Head and Shoulders, Motor Vehicle, Non-motor Vehicle
ROCKIVA_DET_MODEL_CLS8	Model File: object_detection_v3_cls8.data Detection Category: Person, Face, Motor Vehicle, License Plate, Non-motor Vehicle, Pet, Human Head Note: Compared to CLS 7, it is faster, but memory usage will slightly increase.

[Description]

3.1.2.2.9 RockIvaWorkMode

[Function]

Working Mode

[Declaration]

```
typedef enum {
    ROCKIVA_MODE_VIDEO    = 0,
    ROCKIVA_MODE_PICTURE = 1,
} RockIvaWorkMode;
```

[Member]

Member	Description
ROCKIVA_MODE_VIDEO	Video Stream Mode (Enable Tracking)
ROCKIVA_MODE_PICTURE	Image Mode (Disable Tracking)

[Description]

The perimeter function must operate in video stream mode; detection and face functions can use either video stream or image mode;

3.1.2.2.10 RockIvaMemType

[Function]

Memory Type

[Declaration]

```
typedef enum {
    ROCKIVA_MEM_TYPE_CPU = 0,
    ROCKIVA_MEM_TYPE_DMA = 1
} RockIvaMemType;
```

[Member]

Member	Description
ROCKIVA_MEM_TYPE_CPU	Virtual address memory
ROCKIVA_MEM_TYPE_DMA	DMA BUF memory (RV1106)

[Description]

Memory type allocated for image buffers; for the RV1106 platform, DMA BUF memory type must be used.

3.1.2.2.11 RockIvaCameraType

[Function]

Camera Type

[Declaration]

```
typedef enum {
    ROCKIVA_CAMERA_TYPE_ONE = 0,
    ROCKIVA_CAMERA_TYPE_DUAL = 1,
    ROCKIVA_CAMERA_TYPE_SL = 2,
} RockIvaCameraType;
```

[Member]

Member	Description
ROCKIVA_CAMERA_TYPE_ONE	Single camera
ROCKIVA_CAMERA_TYPE_DUAL	Dual camera, capable of providing RGB and IR image streams
ROCKIVA_CAMERA_TYPE_SL	Structured light camera, capable of providing RGB, IR, and Depth image streams

3.1.2.3 Structure

Structure	Description
RockIvaPoint	Point coordinates
RockIvaLine	Line coordinates
RockIvaRectangle	Quadrilateral coordinates
RockIvaQuadrangle	Polygon coordinates
RockIvaArea	Polygon area
RockIvaAreas	Polygon area list
RockIvaSize	Target size
RockIvaAngle	Angle information
RockIvaRectExpandRatio	Configuration of the scale size for the detection box expansion
RockIvaImageInfo	Image information
RockIvaImage	Image
RockIvaObjectInfo	Single Target Detection Result Information
RockIvaDetectResult	Target Detection Result
RockIvaReleaseFrames	Frames List to be Released
RockIvaInitParam	SDK Global Parameter Configuration

3.1.2.3.1 RockIvaPoint

[Function]

Point coordinates

[Declaration]

```
typedef struct {
    uint16_t x;
    uint16_t y;
} RockIvaPoint;
```

[Member]

Member	Description
x	X-coordinate, expressed in parts per ten thousand, valid range 0~9999
y	Y-coordinate, expressed in parts per ten thousand, valid range 0~9999

3.1.2.3.2 RockIvaLine

[Function]

Line coordinates

[Declaration]

```
typedef struct {
    RockIvaPoint head;
    RockIvaPoint tail;
} RockIvaLine;
```

[Member]

Member	Description
head	Head Coordinate
tail	Tail Coordinate

3.1.2.3.3 RockIvaRectangle

[Function]

Quadrilateral coordinates

[Declaration]

```
typedef struct {
    RockIvaPoint topLeft;
    RockIvaPoint bottomRight;
} RockIvaRectangle;
```

[Member]

Member	Description
topLeft	Top Left Coordinate
bottomRight	Bottom Right Coordinate

3.1.2.3.4 RockIvaQuadrangle

[Function]

Arbitrary Quadrilateral Coordinates

[Declaration]

```
typedef struct {
    RockIvaPoint topLeft;
    RockIvaPoint topRight;
    RockIvaPoint bottomLeft;
    RockIvaPoint bottomRight;
} RockIvaQuadrangle;
```

[Member]

Member	Description
topLeft	Top Left Coordinate
topRight	Top Right Coordinate
bottomLeft	Bottom Left Coordinate
bottomRight	Bottom Right Coordinate

3.1.2.3.5 RockIvaArea

[Function]

Polygon Area Coordinates

[Declaration]

```
typedef struct {
    uint32_t pointNum;
    RockIvaPoint points[ROCKIVA_AREA_POINT_NUM_MAX];
} RockIvaArea;
```

[Member]

Member	Description
pointNum	Number of Points
points	Coordinates of All Points Enclosing the Area

3.1.2.3.6 RockIvaAreas

[Function]

Multiple Areas

[Declaration]

```
typedef struct {
    uint32_t areaNum;
    RockIvaArea areas[ROCKIVA_AREA_NUM_MAX];
} RockIvaAreas;
```

[Member]

Member	Description
areaNum	Number of Areas
areas	Region array

3.1.2.3.7 RockIvaSize

[Function]

Target size expansion in all directions

[Declaration]

```
typedef struct {
    uint16_t width;
    uint16_t height;
} RockIvaSize;
```

[Member]

Member	Description
width	Width (parts per ten thousand coordinates, value range 0 ~9999)
height	Height (parts per ten thousand coordinates, value range 0 ~9999)

3.1.2.3.8 RockIvaRectExpandRatio

[Function]

Configuration of the scale size for the detection box expansion

[Declaration]

```
typedef struct {
    float up;
    float down;
    float left;
    float right;
} RockIva RectExpandRatio ;
```

[Member]

Member	Description
up	Proportion of detection box expansion upwards based on box height
down	Proportion of detection box expansion downwards based on box height
left	Proportion of detection box expansion to the left based on box width
right	Proportion of detection box expansion to the right based on box width

3.1.2.3.9 RockIvaAngle

[Function]

Face angle information

[Declaration]

```
typedef struct {
    int16_t pitch;
    int16_t yaw;
    int16_t roll;
} RockIvaAngle;
```

[Member]

Member	Description
pitch	Pitch angle, representing the rotation angle around the x-axis
yaw	Yaw angle, representing the rotation angle around the y-axis
roll	Roll angle, representing the rotation angle around the z-axis

3.1.2.3.10 RockIvaImageInfo

[Function]

Image information

[Declaration]

```
typedef struct {
    uint16_t width;
    uint16_t height;
    RockIvaImageFormat format;
    RockIvaImageTransform transformMode;
} RockIvaImageInfo;
```

[Member]

Member	Description
width	Image width
height	Image Height
format	Image Pixel Format
transformMode	Rotation Mode

3.1.2.3.11 RockIvaImage

[Function]

Image

[Declaration]

```
typedef struct {
    uint32_t frameId;
    uint32_t channelId;
    RockIvaImageInfo info;
    uint32_t size;
    uint8_t* dataAddr;
    uint8_t* dataPhyAddr;
    int32_t dataFd;
    void* extData;
} RockIvaImage;
```

[Member]

Member	Description
frameId	Original Capture Frame Index (User Defined)
channelId	Channel Number (User Defined, set to 0 for single channel)
info	Image information
size	Image Data Size, required when the image format is JPEG
dataAddr	Virtual Address of Input Image Data
dataPhyAddr	Physical Address of Input Image Data (set to NULL if not used)
dataFd	File Handle of Input Image Data (set to 0 if not used)
extData	User Defined Extended Data

3.1.2.3.12 RockIvaObjectInfo

[Function]

Single Target Detection Result Information

[Declaration]

```
typedef struct {
    uint32_t objId;
    uint32_t frameId;
    uint32_t score;
    RockIvaRectangle rect;
    RockIvaObjectType type;
} RockIvaObjectInfo;
```

[Member]

Member	Description
objId	Target ID[0~2 ³²)
frameId	Frame ID
score	Object Detection Score [1-100]
rect	Target Area Box (Parts per ten thousand)
type	Target Category

3.1.2.3.13 RockIvaDetectResult

[Function]

Target Detection Result

[Declaration]

```
typedef struct {
    uint32_t frameId;
    RockIvaImage frame;
    uint32_t channelId;
    uint32_t objNum;
    RockIvaObjectInfo objInfo[ROCKIVA_MAX_OBJ_NUM];
} RockIvaDetectResult;
```

[Member]

Member	Description
frameId	Frame ID
frame	Corresponding Input Image Frame
channelId	Channel ID
objNum	Number of Targets
objInfo	Detection Information for Each Target

3.1.2.3.14 RockIvaReleaseFrames

[Function]

List of frames that can be released

[Declaration]

```
typedef struct {
    uint32_t channelId;
    uint32_t count;
    RockIvaImage frame Id [ROCKIVA_MAX_OBJ_NUM];
} RockIvaReleaseFrames;
```

[Member]

Member	Description
channelId	Channel ID
count	Quantity
frame Id	Releasable frames

3.1.2.3.15 RockIvaMediaOps

[Function]

Multimedia operation functions

[Declaration]

```
typedef struct {
    int (*ROCKIVA_Yuv2Jpeg) (RockIvaImage *inputImg, RockIvaImage
*outputImg, void *userdata);
} RockIvaMediaOps;
```

[Member]

Member	Description
ROCKIVA_Yuv2Jpeg	Encode image to JPEG image

3.1.2.3.16 RockIvaInitParam

[Function]

Global algorithm parameter configuration

[Declaration]

```
typedef struct {
    RockIvaLogLevel logLevel;
    char logPath[ROCKIVA_PATH_LENGTH];
    char modelPath[ROCKIVA_PATH_LENGTH];
    RockIvaMemInfo license;
    uint32_t coreMask;
    uint32_t channelId;
    RockIvaImageInfo imageInfo;
    RockIvaAreas roiAreas;
    RockIvaCameraType cameraType;
    RockIvaDetModel detModel;
    uint32_t detClasses;
    RockIvaMemInfo detModelData;
    uint8_t trackerVersion;
    RockIvaMediaOps mediaOps;
} RockIvaInitParam;
```

[Member]

Member	Description
RockIvaLogLevel	Log level
logPath	Log output path
modelPath	Path to store algorithm model
license	License information
coreMask	Specify which NPU core to run (valid only for RK3588 platform)
channelId	Channel number
imageInfo	Input image information
roiAreas	Valid area
cameraType	Camera Type
detModel	Pre-stage object detection model
detClasses	No configuration is needed under normal circumstances; only some special detection categories require configuration, such as baby detection: ROCKIVA_OBJECT_TYPE_BITMASK(ROCKIVA_OBJECT_TYPE_BABY)
detModelData	Specify detection model data (for quick start, directly configure model memory data for loading)
detModelName	Specify detection model name
trackerVersion	Specify Tracking Algorithm Version (Deprecated)
mediaOps	Set Media Operation Callback Function

3.2 Detection module

3.2.1 Function Description

The detection model primarily obtains information such as the bounding box coordinates and scores of the detected targets in the image. The corresponding header file for the detection module is `rockiva_det_api.h`, and you can configure the initialization parameters and set the detection result callback through `ROCKIVA_DETECT_Init`. The basic calling process is shown in Figure 3-2.

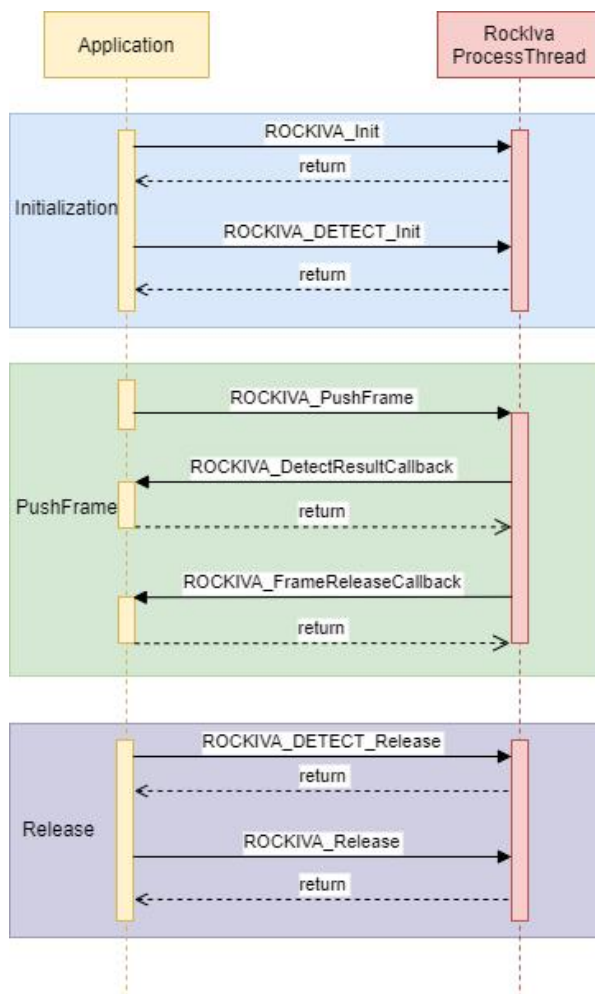


Figure 3-3 Detection Function Calling Process

After configuration initialization, the SDK will create threads internally to asynchronously process each frame. Each image frame pushed by the application will be placed into a cache queue for the processing thread. It should be noted that the result callback function `ROCKIVA_DetectResultCallback` runs in the internal processing thread, so try to avoid performing operations that take too long, as this may affect the processing frame rate.

The detection module can operate in video stream mode or image mode. In video stream mode, target tracking is enabled, and the `objId` of each detected target in the detection callback is valid; In image mode, target tracking is disabled, and the `objId` value of detected targets is invalid.

For each frame pushed by the user, the result is also released through the `ROCKIVA_FrameReleaseCallback` callback function.

The following is a reference code example for the detection module:


```

RockIvaRetCode ret;
RockIvaHandle handle;

// Release callback function (same as the general module
example code)
...

// Detection module callback function
void DetResultCallback(const RockIvaDetectResult* result, const
RockIvaExecuteStatus status, void* userdata)
{
    // Handle results
}

// Initialize I VA instance
RockIvaInitParam commonParams ;
memset (&commonParams, 0, sizeof(RockIvaInitParam));
commonParams.detModelName =
iva_object_detection_v3_cls8_896x512.data; // Set detection model
v ret = ROCKIVA_Init(&handle, ROCKIVA_MODE_VIDEO, &commonParams,
NULL);
// Set Callback Function
ROCKIVA_SetFrameReleaseCallback(handle, framerelease_callback);

// Initialize detection module
RockIvaDetTaskParams detParams;
memset (&detParams, 0, sizeof(RockIvaDetTaskParams));
// Set Report Target Type
detParams->detObjectType |=
ROCKIVA_OBJECT_TYPE_BITMASK(ROCKIVA_OBJECT_TYPE_PERSON)
// Set Detection Score Threshold, setting only the 0th can take
effect for all categories
detParams->scores[0] = 30;
ret = ROCKIVA_DETECT_Init(handle, &detParams, DetResultCallback);

// Frame Processing (same as the general module example code)
...

// Wait for all frames to be processed
usleep(5*1000*1000);

```

```
// Destroy Detection Module
ROCKIVA_DETECT_Release(handle)
// Destroy IVA Instance
ROCKIVA_Release(handle);
```

3.2.2 API Reference

3.2.2.1 Function Interface

Interface	Description
ROCKIVA_DetectResultCallback	Result Callback Function
ROCKIVA_DETECT_Init	Initialization
ROCKIVA_DETECT_Release	Release
ROCKIVA_DETECT_Reset	Runtime Reconfiguration (reconfiguration will cause some internal records to be cleared and reset, but the model will not be reinitialized)

3.2.2.1.1 ROCKIVA_DetectResultCallback

[Function]

Result Callback Function

[Declaration]

```
typedef void (* ROCKIVA_DetectResultCallback ) (
    const RockIvaDetectResult* result,
    const RockIvaExecuteStatus status,
    void* userdata);
```

[Input Parameters]

Parameter Name	Input /Output	Description
result	Input	Result
status	Input	Status Code
userdata	Input	User-defined data

[Return Value]

RockIvaRetCode

3.2.2.1.2 ROCKIVA_DETECT_Init

[Function]

Initialize Detection Module

[Declaration]

```
RockIvaRetCode ROCKIVA_DETECT_Init(RockIvaHandle handle,
                                   const RockIvaDetTaskParams* params,
                                   const ROCKIVA_DetectResultCallback
resultCallback);
```

[Input Parameters]

Parameter Name	Input /Output	Description
handle	Input	Instance Handle
params	Input	Initialization Parameters
resultCallback	Input	Result Callback Function

[Return Value]

RockIvaRetCode

3.2.2.1.3 ROCKIVA_DETECT_Release

[Function]

Release

[Declaration]

```
RockIvaRetCode ROCKIVA_DETECT_Release (RockIvaHandle handle);
```

[Input Parameters]

Parameter Name	Input /Output	Description
handle	Input	Instance Handle

[Return Value]

RockIvaRetCode

3.2.2.1.4 ROCKIVA_DETECT_Reset

[Function]

Runtime Reconfiguration (reconfiguration will cause some internal records to be cleared and reset, but the model will not be reinitialized)

[Declaration]

```
RockIvaRetCode ROCKIVA_DETECT_Reset(RockIvaHandle handle ,
                                   RockIvaDetTaskParams* params );
```

[Input Parameters]

Parameter Name	Input /Output	Description
handle	Input	Handle to be reset, runtime reconfiguration (reconfiguration will cause some internal records to be cleared and reset, but the model will not be reinitialized)
params	Input	Initialization Parameters

[Return Value]

RockIvaRetCode

3.2.2.2 Structure

Structure	Description
RockIvaDetTaskParams	Object Detection Business Initialization Parameter Configuration

3.2.2.2.1 RockIvaDetTaskParams

[Function]

Object Detection Business Initialization Parameter Configuration

[Declaration]

```
typedef struct {
    uint32_t detObjectType;
    RockIvaAreas roiAreas;
    uint8_t scores[ROCKIVA_OBJECT_TYPE_MAX];
    uint8_t min_det_count;
} RockIvaDetTaskParams;
```

[Member]

Member	Description
detObjectType	Configure the detection targets to be returned
roiAreas	Configure the valid detection area (only affects detection results)
scores	Score threshold for filtering each category, 0 for internal automatic
min_det_count	Report detection targets only after the stable detection frame count exceeds this value, filtering out some low-probability false positives

3.3 Perimeter module

3.3.1 Function Description

The process of the perimeter function is basically similar to detection, initializing perimeter-related configurations and setting

up perimeter result callbacks through ROCKIVA_BA_Init.

The code calling process of the perimeter function module is shown in Figure 3-4.

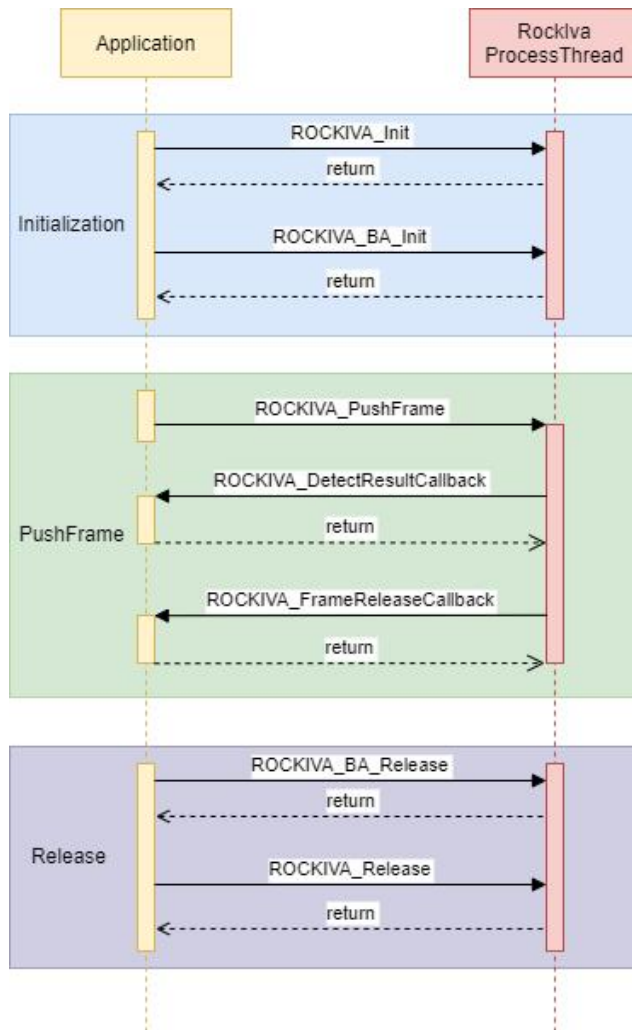


Figure 3-4 Perimeter Function Calling Process

Note that the working mode of the perimeter must be video stream mode (ROCKIVA_MODE_VIDEO), as the rule judgment of the perimeter needs to be calculated based on the results of the target's previous and subsequent frames.

There are four types of perimeter functions:

- > Area intrusion: Triggered when the target enters the designated area and stays for longer than the set time
- > Area entry: Triggered when the target enters the area from outside the designated area
- > Area exit: Triggered when the target leaves the area to outside the designated area
- > Boundary crossing detection: Triggered when the target crosses the designated line segment (in the specified direction)

Configurable rules mainly include:

Rule configuration	Description
Area	Set rule area: A polygon formed by a maximum of 6 ordered points
Boundary line	Set rule boundary line: A line segment formed by two points, direction can be set
Target type filtering	Set the target types that trigger: Person, vehicle, non-motor vehicle
Target size filtering	Set target size: minimum and maximum width and height

The area intrusion results are shown in Figures 3-5 and 3-6. An event will be reported when the target enters the restricted area.

The boundary crossing results are shown in Figure 3-7. An event will be reported when the target crosses the boundary line from a prohibited direction.



Figure 3-5 Area intrusion before entering the restricted area

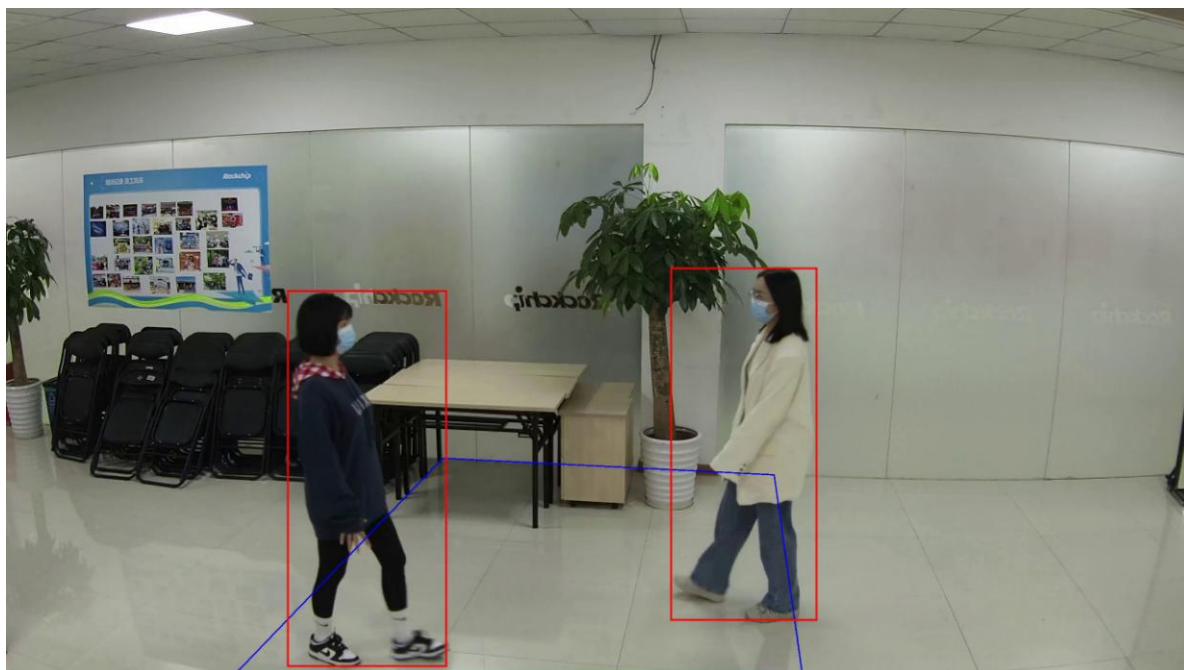


Figure 3-6 Area intrusion after entering the restricted area

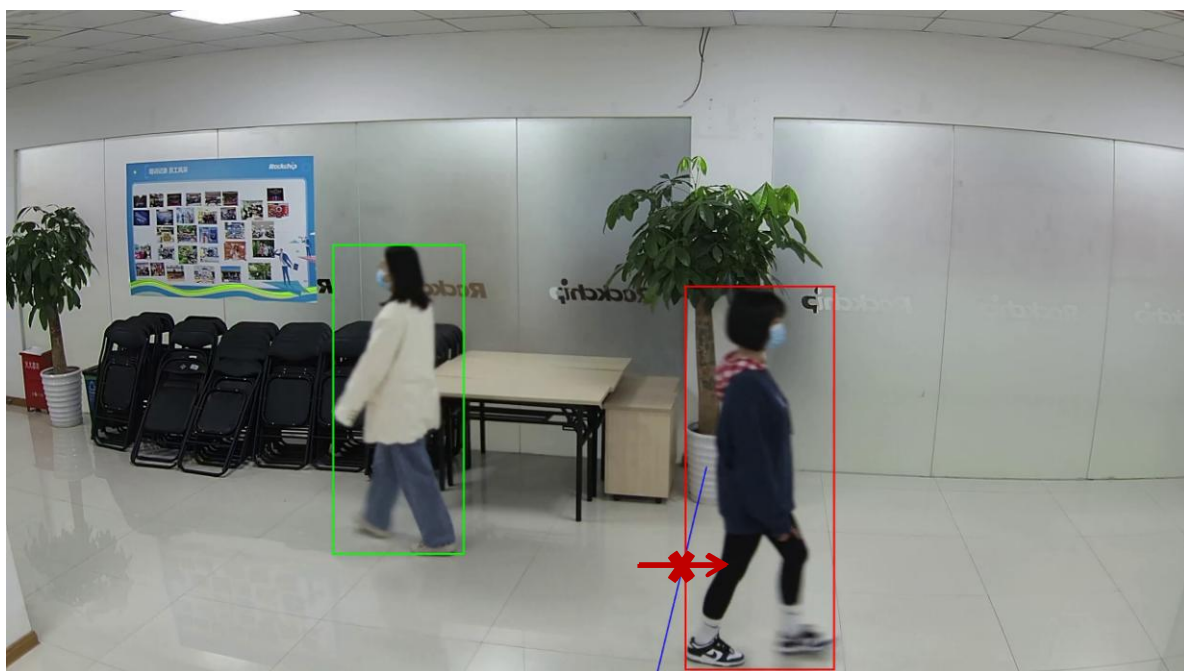


Figure 3-7 Boundary crossing results

The following is the reference code related to the perimeter module

```

RockIvaRetCode ret;
RockIvaHandle handle;

// Release callback function (same as the general module
example code)
...

// Perimeter module callback function
void BaResultCallback(const RockIvaBaResult* result, const
RockIvaExecuteStatus status, void* userdata)
{
    // Handle results
}

// Initialize IVA instance
RockIvaInitParam commonParams ;
memset (&commonParams, 0, sizeof(RockIvaInitParam));
commonParams.detModelName =
iva_object_detection_v3_cls8_896x512.data; // Set detection model

ret = ROCKIVA_Init(&handle, ROCKIVA_MODE_VIDEO, &commonParams,
NULL);
// Set Callback Function
ROCKIVA_SetFrameReleaseCallback(handle, framerelease_callback);

// Initialize perimeter module
RockIvaBaTaskParams baParams ;
memset (&baParams, 0, sizeof(RockIvaBaTaskParams));

// Build an area intrusion rule
baParams.baRules.areaInBreakRule[0].ruleEnable = 1;
baParams.baRules.areaInBreakRule[0].alertTime = 1000; //
Trigger time for target within area
baParams.baRules.areaInBreakRule[0].ruleID = 1;
baParams.baRules.areaInBreakRule[0].objType |=
ROCKIVA_OBJECT_TYPE_BITMASK(ROCKIVA_OBJECT_TYPE_PE
RSON);
// The area is formed by connecting multiple points in order
baParams.baRules.areaInBreakRule[0].area.pointNum = 4;
baParams.baRules.areaInBreakRule[0].area.points[0].x =1000;

```



```

baParams.baRules.areaInBreakRule[0].area.points[0].y =1000;
baParams.baRules.areaInBreakRule[0].area.points[1].x =9000;
baParams.baRules.areaInBreakRule[0].area.points[1].y = 1000;
baParams.baRules.areaInBreakRule[0].area.points[2].x = 9000;
baParams.baRules.areaInBreakRule[0].area.points[2].y =9000;
baParams.baRules.areaInBreakRule[0].area.points[3].x =1000;
baParams.baRules.areaInBreakRule[0].area.points[3].y = 9000;

// Build a boundary crossing rule
baParams.baRules.tripWireRule[0].ruleEnable = 1;
baParams.baRules.tripWireRule[0].event =
ROCKIVA_BA_TRIP_EVENT_BOTH;
baParams.baRules.tripWireRule[0].ruleID = 2;
baParams.baRules.tripWireRule[0].objType |=
    ROCKIVA_OBJECT_TYPE_BITMASK(ROCKIVA_OBJECT_TYPE_PERSON);

baParams.baRules.tripWireRule[0].line.head.x = 1000;
baParams.baRules.tripWireRule[0].line.head.y =5000;
baParams.baRules.tripWireRule[0].line.tail.x = 9000;
baParams.baRules.tripWireRule[0].line.tail.y =5000;

ret = ROCKIVA_BA_Init(handle, &baParams, BaResultCallback);

// Frame Processing (same as the general module example code)
...

// Wait for all frames to be processed
usleep(5*1000*1000);

// Destroy perimeter module
ROCKIVA_BA_Release(handle)
// Destroy IVA Instance
ROCKIVA_Release(handle);

```

3.3.2 API Reference

3.3.2.1 Function Interface

Interface	Description
-----------	-------------

ROCKIVA_BA_ResultCallback	Result Callback Function
ROCKIVA_BA_Init	Initialization
ROCKIVA_BA_Destroy	Destroy
ROCKIVA_BA_Reset	Reset

3.3.2.1.1 ROCKIVA_BA_ResultCallback

[Function]

Result Callback Function

[Declaration]

```
typedef void(* ROCKIVA_BA_ResultCallback ) (
    const RockIvaBaResult *result,
    const RockIvaExecuteStatus status,
    void * userdata );
```

[Input Parameters]

Parameter Name	Input /Output	Description
result	Input	Result
status	Input	Status Code
userdata	Input	User-defined data

[Return Value]

RockIvaRetCode

3.3.2.1.2 ROCKIVA_BA_Init

[Function]

Initialization

[Declaration]

```
RockIvaRetCode ROCKIVA_BA_Init (RockIvaHandle handle,
    const RockIvaBaTaskParam s *initParams,
    const ROCKIVA_BA_ResultCallback
resultCallback);
```

[Input Parameters]

Parameter Name	Input /Output	Description
handle	Input	Handle to be initialized
initParams	Input	Initialization parameter configuration
resultCallback	Input	Callback Function

[Return Value]

RockIvaRetCode

3.3.2.1.3 ROCKIVA_BA_Destroy

[Function]

Destroy

[Declaration]

```
RockIvaRetCode ROCKIVA_BA_Destroy (RockIvaHandle handle);
```

[Input Parameters]

Parameter Name	Input /Output	Description
handle	Input	Handle to be destroyed

[Return Value]

RockIvaRetCode

3.3.2.1.4 ROCKIVA_BA_Reset

[Function]

Reset

[Declaration]

```
RockIvaRetCode ROCKIVA_BA_Reset (RockIvaHandle handle, const
RockIvaBaTaskParams* params );
```

[Input Parameters]

Parameter Name	Input /Output	Description
handle	Input	Handle to be reset
params	Input	New initialization parameter configuration

[Return Value]

RockIvaRetCode

3.3.2.2 Enumeration definition

Enumeration	Description
RockIvaBaTripEvent	Perimeter rule type (tripwire/area event)
RockIvaBaRuleTriggerType	Target rule trigger type

3.3.2.2.1 RockIvaBaTripEvent

[Function]

Perimeter rule tripwire event type

[Declaration]

```
typedef enum {
    ROCKIVA_BA_TRIP_EVENT_BOTH = 0,
    ROCKIVA_BA_TRIP_EVENT_DEASIL = 1,
    ROCKIVA_BA_TRIP_EVENT_WIDDERSHINES = 2,
} RockIvaBaTripEvent;
```

[Member]

Member	Description
ROCKIVA_BA_TRIP_EVENT_BOTH	Tripwire: Bidirectional trigger
ROCKIVA_BA_TRIP_EVENT_DEASIL	Tripwire: Clockwise trigger
ROCKIVA_BA_TRIP_EVENT_WIDDERSHINES	Tripwire: Counterclockwise trigger

3.3.2.2.2 RockIvaBaRuleTriggerType

[Function]

Target rule trigger type

[Declaration]

```
typedef enum {
    ROCKIVA_BA_RULE_NONE = 0,
    ROCKIVA_BA_RULE_CROSS = 0 x 1,
    ROCKIVA_BA_RULE_INAREA = 0 x2,
    ROCKIVA_BA_RULE_OUTAREA = 0 x4,
    ROCKIVA_BA_RULE_STAY = 0 x8,
} RockIvaBaRuleTriggerType;
```

[Member]

Member	Description
ROCKIVA_BA_RULE_NONE	
ROCKIVA_BA_RULE_CROSS	Line Mixing
ROCKIVA_BA_RULE_INAREA	Enter Area
ROCKIVA_BA_RULE_OUTAREA	Leave Area
ROCKIVA_BA_RULE_STAY	Area intrusion

3.3.2.3 Structure Definition

Structure	Description
RockIvaBaWireRule	Boundary Crossing Rule
RockIvaBaAreaRule	Area Rule
RockIvaBaTaskRule	Behavior Analysis Rule Configuration
RockIvaBaAiConfig	Algorithm Configuration
RockIvaBaTaskParams	Behavior Analysis Business Initialization Parameters

RockIvaBaTrigger	First Trigger Rule Information
RockIvaBaObjectInfo	Basic Information of Single Target Detection
RockIvaBaResult	Complete Detection Result Information

3.3.2.3.1 RockIvaBaWireRule

[Function]

Boundary Crossing Rule

[Declaration]

```
typedef struct {
    uint8_t ruleEnable;
    uint32_t ruleID;
    RockIvaLine line;
    RockIvaBaTripEvent event;
    RockIvaSize minObjSize[ROCKIVA_OBJECT_TYPE_MAX];
    RockIvaSize maxObjSize[ROCKIVA_OBJECT_TYPE_MAX];
    uint8_t scores[ROCKIVA_OBJECT_TYPE_MAX];
    uint32_t objType;
    uint8_t rulePriority;
    uint8_t sense;
    uint8_t triggerMode;
} RockIvaBaWireRule;
```

[Member]

Member	Description
ruleEnable	Whether the rule is enabled, 1-> enabled, 0-> not enabled
ruleID	Rule ID, valid range [0, 3]
line	Boundary Crossing Line Configuration
event	Boundary Crossing Direction
minObjSize	Parts per ten thousand indicates Minimum Target: 0 Motor Vehicle 1 Non-motor Vehicle 2 Pedestrian
maxObjSize	Parts per ten thousand indicates Maximum Target: 0 Motor Vehicle 1 Non-motor Vehicle 2 Pedestrian
scores	Score threshold for filtering each category, 0 for internal automatic
objType	Configure trigger target types: for example
rulePriority	Rule priority: 0 High, 1 Medium, 2 Low
sense	Sensitivity, 1~100
triggerMode	Trigger mode: 0: Target triggers only once; 1: Target triggers every time it crosses the boundary

3.3.2.3.2 RockIvaBaAreaRule

[Function]

Area Rule

[Declaration]

```
typedef struct {
    uint8_t ruleEnable;
    uint32_t ruleID;
    RockIvaArea area;
    RockIvaSize minObjSize[ROCKIVA_OBJECT_TYPE_MAX];
    RockIvaSize maxObjSize[ROCKIVA_OBJECT_TYPE_MAX];
    uint8_t scores[ROCKIVA_OBJECT_TYPE_MAX];
    uint32_t objType;
    uint32_t alertTime;
    uint8_t sense;
    uint8_t checkEnter;
} RockIvaBaAreaRule;
```

[Member]

Member	Description
ruleEnable	Whether the rule is enabled, 1-> enabled, 0-> not enabled
ruleID	Rule ID, valid range [1, 256]
area	Area configuration
minObjSize	Parts per ten thousand indicates Minimum Target: 0 Motor Vehicle 1 Non-motor Vehicle 2 Pedestrian
maxObjSize	Parts per ten thousand indicates Maximum Target: 0 Motor Vehicle 1 Non-motor Vehicle 2 Pedestrian
scores	Score threshold for filtering each category, 0 for internal automatic
objType	Configure trigger target types
alertTime	Area intrusion rule alarm time setting
sense	Sensitivity, 1~100
checkEnter	Does the area intrusion rule need to check if the target has entered [0: Not enabled, 1: Enabled]

3.3.2.3.3 RockIvaBaTaskRule

[Function]

Perimeter rule configuration

[Declaration]

```
typedef struct {
    RockIvaBaWireRule tripWireRule[ROCKIVA_BA_MAX_RULE_NUM];
    RockIvaBaAreaRule areaInRule[ROCKIVA_BA_MAX_RULE_NUM];
    RockIvaBaAreaRule areaOutRule[ROCKIVA_BA_MAX_RULE_NUM];
    RockIvaBaAreaRule areaInBreakRule[ROCKIVA_BA_MAX_RULE_NUM];
} RockIvaBaTaskRule;
```

[Member]

Member	Description
tripWireRule	Boundary crossing event
areaInRule	Enter Area
areaOutRule	Leave Area
areaInBreakRule	Area intrusion

3.3.2.3.4 RockIvaBaAiConfig

[Function]

Algorithm Configuration

[Declaration]

```
typedef struct {
    uint8_t filterPersonMode;
    uint8_t detectResultMode;
} RockIvaBaAiConfig;
```

[Member]

Member	Description
filterPersonMode	Filter non-motor vehicle/motor vehicle driver person results 0: Do not filter; 1: Filter
detectResultMode	Reporting Object Detection Results Mode 0: Do not report detected targets that did not trigger rules; 1: Report detected targets that did not trigger rules

3.3.2.3.5 RockIvaBaTaskParam s

[Function]

Perimeter Initialization Parameter Configuration

[Declaration]

```
typedef struct {  
    RockIvaBaTaskRule baRules;  
    RockIvaBaAiConfig aiConfig;  
} RockIvaBaTaskParam s ;
```

[Member]

Member	Description
baRules	Perimeter Rule Parameter Configuration Initialization
aiConfig	Algorithm Configuration

3.3.2.3.6 RockIvaBaTrigger

[Function]

Information about the first time the target triggered a rule, which can be used for snapshot

[Declaration]

```
typedef struct {  
    int32_t ruleID;  
    RockIvaBaRuleTriggerType triggerType;  
} RockIvaBaTrigger;
```

[Member]

Member	Description
ruleID	Triggered Rule ID
triggerType	Triggered Rule Type

3.3.2.3.7 RockIva Ba ObjectInfo

[Function]

Information about a single target that triggered the perimeter rule

[Declaration]


```
typedef struct {
    RockIvaObjectInfo objInfo;
    uint32_t triggerRules;
    uint32_t triggerRulesNum;
    uint32_t triggerRulesID[ROCKIVA_BA_MAX_RULE_NUM];
    RockIvaBaTrigger firstTrigger;
} RockIvaBaObjectInfo;
```

[Member]

Member	Description
objInfo	Object Detection Result Information
triggerRules	Rules Triggered by the Target
triggerRulesNum	Number of Rules Triggered by the Target
triggerRulesID	All Rule IDs Triggered by the Target
firstTrigger	The Rule Triggered by the Target for the First Time

3.3.2.3.8 RockIvaBaResult

[Function]

Complete Detection Result Information

[Declaration]

```
typedef struct {
    uint32_t frameId;
    RockIvaImage frame;
    uint32_t channelId;
    uint32_t objNum;
    RockIvaObjectInfo triggerObjects [ROCKIVA_MAX_OBJ_NUM];
} RockIvaBaResult;
```

[Member]

Member	Description
frameId	Input Image Frame ID
frame	Corresponding Input Image Frame
channelId	Channel number
objNum	Number of Targets
triggerObjects	Target that Triggered the Perimeter Rule